

Heather Carminati

CS5100 Foundations of Artificial Intelligence

April 12, 2026

Final Report

Abstract

This paper presents an AI system for playing Stool Pigeon, a competitive memory-based card game where both players have hidden cards and must take decisions under uncertainty. Four agents of increasing sophistication were implemented, including a random baseline, a rule-based heuristic agent, a Partially Observable Markov Decision Process (POMDP) agent that maintains a probability distribution over hidden cards and updates it as the game progresses, and a Monte Carlo-augmented POMDP agent that estimates action values through game simulation. This approach is adapted from Yao et al. (2020), who applied POMDP and Monte Carlo methods to Rhode Island Hold'em poker. All four agents were evaluated against each other in 500 games per matchup and win rate, average game length, average final score, knock accuracy, and belief state error were measured. The POMDP agent won 79.0% of games against the random agent and 81.6% against the heuristic agent with a knock accuracy of 91.3% and 83.2% respectively. The MC+POMDP agent won 93.6% against the heuristic agent and 95.4% against the basic POMDP agent with a knock accuracy of 93.8% and 96.7%. These results demonstrate that Monte Carlo simulation adds significant value on top of belief-state reasoning alone. A live, playable version of the game is available at <https://stool-pigeon.netlify.app>, and the full source code is available at <https://github.com/hcarminati/Stool-Pidgeon>.

Introduction

Imperfect information games are among the most studied problems in artificial intelligence. Unlike perfect information games like chess where every player can see the entire game state, imperfect information games require agents to make decisions without full knowledge of what their opponents hold or what cards remain hidden (Russell & Norvig, 2020). This uncertainty makes naive search approaches impractical and requires agents to reason probability over possible game states. Examples of imperfect information games include poker and bridge, but the same challenge appears in many other card games that combine memory, inference, and strategic decision-making.

This project focuses on Stool Pigeon, a competitive card game inspired by Golf for two to six players where each player maintains a “crime scene” of four face-down cards and tries to minimize their total score (Miller, 2023). Players begin by privately viewing two of their four face-down cards. Then they take turns drawing cards and deciding whether to keep or discard them. The player with the lowest total card value when the round ends wins. Several features make Stool Pidgeon an interesting AI challenge. First, both players have hidden cards, meaning that unlike poker, where only the opponent’s hand is concealed, the agent must maintain uncertainty about its own unseen cards as well. Second, the game includes four special action cards, Stool Pigeon (peek at any face-down card), Bamboozle (swap any two face-down cards), Vendetta (peek then swap), and Kingpin (remove a card from your crime scene or force the opponent to add one to their crime scene). Each of these special cards creates distinct information events that must be tracked. Third, a player can choose to “knock” at any point during their turn. Knocking ends the round after one final turn for each opponent, which adds a timing dimension to the strategy. For the purpose of this project, the witness-card real-time elimination mechanic from the official rules was excluded to keep the focus on strategic reasoning rather than reaction speed.

The central research question of this project is how well a Partially Observable Markov Decision Process (POMDP) framework, combined with Monte Carlo simulation, can support effective play in Stool Pigeon compared to simpler agent designs. Yao et al. (2020) addresses the same class of problems in the context of Rhode Island Hold'em poker. In that paper, the author models the poker game as a POMDP where the agent maintains a probability distribution over hidden opponent cards and uses Monte Carlo sampling to estimate action values without requiring explicit enumeration of the full game tree. They demonstrate that this approach produces reliable estimates and outperforms simpler baselines. This project adapts and extends those ideas to Stool pigeon which has a more complex hidden information structure than poker due to the mutual uncertainty over both players' hands and the variety of special card interactions.

This project delivers four agents of increasing sophistication, a random agent, a rule-based heuristic agent, a basic POMDP agent, and a Monte Carlo-augmented POMDP agent. It also delivers a fully playable game implementation and a systematic evaluation framework. The agents are evaluated against one another in 500-game matchups to measure the contribution of each layer of reasoning.

Methods

Game Environment

The game was implemented from scratch in Python using PyGame (Pygame Community, 2024). The implementation supports both a graphical user interface (GUI) mode where a human player can play against one of any of the four agents and a headless mode used for automated agent versus agent evaluation. The game environment handles all card types (numbered cards 2–10, Meatball worth zero points, Rat worth fifteen points and removable only by Kingpin, and the four

action cards), a complete action system, game phase tracking, scoring, and game over detection.

Game state is managed through GameState class that tracks the current player, the current phase (title screen, start, draw, decide, special card phases, final turn, and game over), which player knocked, and any card currently drawn but not yet placed. Phases transition deterministically in response to actions and the phase system makes sure that both the human player and AI agents interact with the game through the same action interface. This unified design was important because it meant that every action has to work correctly in two contexts, where the user input drives transitions and where the agent executes actions immediately.

Actions are represented as objects with a type (draw from pile, draw from discard, keep card, discard drawn, knock, peek, swap, done peeking, kingpin eliminate, and kingpin add) and optional target fields for certain actions. Each action's execute_action method modifies the same state directly and keeps the action logic centralized for consistency across GUI and headless modes. Legal actions for the current player are generated dynamically based on the current phase and game state.

The GUI uses a dark color scheme with a red-orange accent color. The background changes to indicate whose turn it is, and each special card triggers a distinct multi-phase interaction flow with hover effects and phase-specific click handling. A title screen presents the game rules and allows the player to select a difficulty level (Easy, Medium, Hard, or Expert) that maps to the random, heuristic, POMDP, and MC+POMDP agents respectively. An end game screen displays scores and allows the player to start a new game.

Agent 1: Random Agent

The random agent serves as the weakest baseline. At each decision point, it retrieves the list of all legal actions from the game and selects one at random. It maintains no memory of past events and makes no use of the visible game state. Despite its simplicity, it is a useful baseline

because it provides a lower bound on performance that any reasoning agent should exceed. Wins against the random agent demonstrate that an agent's strategy adds value beyond chance.

Agent 2: Heuristic Agent

The heuristic agent applies a fixed set of rules. For draw decisions, it takes from the discard pile if the top card's values are below five, otherwise it draws from the draw pile. For keep decisions, it keeps a drawn card if its value is below five and places it into the slot with the highest known kept value to maximize improvement, otherwise it discards the drawn card. For knock decisions, it knocks if the sum of values across slots it has personally drawn and kept is at or below twelve. For special actions (peek, swap, and kingpin), it has no basis to reason and selects randomly. The agent maintains a running tally of the slots it has actively placed cards into, which gives it a partial memory of its own hand, but it has no knowledge of slots it has not personally modified, no model of the opponent's hand, and no tracking of hidden information beyond its own kept cards.

Agent 3: Basic POMDP Agent

The basic POMDP agent formalizes the game as a Partially Observable Markov Decision Process following the approach described in Yao et al. (2020). At any point during the game, the true state, the identity of every hidden card, is not fully observable. The agent maintains a belief state that tracks this uncertainty and updates it incrementally as new information is revealed.

The belief state uses two data structures. The first is a dictionary that maps each (player, slot) pair to a card object if the card at that position is known, or null if it is not. The second is a counter that tracks the multiset of all cards not yet accounted for. At game state, the belief state is initialized from the full deck, and the agent's own two face-up cards are immediately marked

as known and removed from the unknown pool. As the game progresses, observations update the belief state in different ways depending on the type of event. When the agent peeks at a card with the Stool Pigeon or Vendetta, that card is marked as known. When the agent keeps a drawn card, the known card at that slot is updated to the drawn card, and the displaced card, which is discarded face-up, is removed from the game entirely. When any card is discarded, it is removed from the unknown pool. When a swap occurs via Bamboozle or Vendetta, the belief state propagates known cards across slots; if the card at one slot was known and the other was not, the known card moves to the new slot and the old slot becomes uncertain. If both were known, they simply exchange. Kingpin eliminations mark the removed card as gone from the game.

These updates are handled by a separate Observation data class. Each game event creates an observation with a type and relevant fields, and calling `update_belief` on the observation applies the appropriate change to the belief state. This incremental update system closely mirrors the Bayesian updating approach described in Yao et al. (2020), where each observable action refines the agent's estimate of the hidden state.

Using the belief state, the agent computes expected values for each slot. For known slots, the expected value is simply the card's actual value. For unknown slots, the expected value is the probability-weighted average over all remaining unknown cards, the sum over all card types of their point value multiplied by the fraction of the unknown pool they represent. As cards are directly observed through peeks, discards, and keep events, they are removed from the unknown pool and probability is redistributed uniformly over the remaining unaccounted-for cards. Because of this the agent's uncertainty about hidden cards narrows as more evidence accumulates over the course of a game. When an opponent draws from the pile and keeps it, the drawn card is never revealed, and no inference is made that it was probably low-valued. The only updates that occur are from directly observable events, e.g. cards that are shown face-up

when discarded, peeked, or displaced by a keep. The total expected hand value for a player is the sum of expected values across all their active slots.

The agent's decision logic uses this expected value to select actions. For draw decisions, it compares the top of the discard pile to the maximum expected slot value in its hand, and only draws from the discard pile if the known card would improve its hand. For keep decisions, it computes the expected hand value after each possible swap and keeps the card only if doing so reduces the total. For peek actions triggered by Stool Pigeon, it targets the opponent's unknown slot with the highest expected value. For swap actions triggered by Bamboozle or Vendetta, it selects the swap that maximizes the difference between the expected value of its own slot being sent away and the expected value of the opponent's slot coming in. For Kingpin, it eliminates its own highest expected value slot or adds a card to the opponent if the opponent is winning. The agent knocks when its expected hand value falls below a threshold of fifteen.

Agent 4: MC+POMDP Agent

The Monte Carlo POMDP agent extends the basic POMDP agent by replacing its heuristic draw and knock decisions with simulation-based estimates, which also follows the Monte Carlo Optimization approach from Yao et al. (2020). Rather than comparing the discard pile's top card to the worst expected slot value, or knocking based on a fixed score threshold, it estimates the downstream value of each candidate action by simulating how the rest of the game plays out.

For each candidate action, the agent runs one hundred simulations. In each simulation, it samples a concrete game state from the belief distribution using `sample_game_state` method, which assigns a specific card to each unknown slot by drawing without replacement from the unknown pool. It then applies the candidate action to a deep copy of the game and simulates the rest of the game to completion using a lightweight rollout policy. The rollout policy is not the full POMDP decision logic but a fast greedy rule set; draw from the discard if the top card is

below six, keep a drawn card only if it beats the current worst slot, peek at the first available target, swap at the first available swap, and knock if the hand score is below fifteen. The simulation returns one if the agent wins, negative one if it loses, and zero for a tie. After one hundred simulations, the average reward for each candidate action is computed, and the action with the highest average is selected. Yao et al. (2020) proves that this Monte Carlo estimator is unbiased, the expected value of the sample average is proportional to the true action value, which means that with enough simulations the estimates converge to the correct ranking of actions.

For the knock decision specifically, the agent compares the simulated value of knocking against all available non-knock alternatives and knocks only if the knock action has the highest estimated return. This replaces the inherited fixed threshold from the basic POMDP agent with a multi-step lookahead that accounts for how the current hand compares to what could be achieved by continuing to draw, at the cost of running simulations on every knock decision. All other decision types (keep, peek, swap, kingpin) are inherited unchanged from the basic POMDP agent. This design follows the same structure as Yao et al. (2020), where Monte Carlo sampling is applied to specific decision bottlenecks rather than replacing the entire policy.

Evaluation

Agent performance was evaluated using a headless evaluation script that runs games between pairs of agents in parallel using Python's multiprocessing module with ten worker processes (Python Software Foundation, 2024). Win rate statistics and belief state error were computed using NumPy (Harris et al., 2020). Six matchups were selected and each ran for five hundred games. The matchups answer specific questions about each layer of reasoning: Heuristic vs. Random establishes whether rule-based decisions add value over chance; POMDP vs. Random tests whether belief-state reasoning improves on no reasoning; POMDP vs. Heuristic tests

belief-state reasoning against a stronger baseline; MC+POMDP vs. Random and MC+POMDP vs. Heuristic measure the full system against both baselines; and MC+POMDP vs. POMDP isolates the specific contribution of the Monte Carlo layer by holding the belief state constant.

All opponents use fixed and pre-defined policies and do not adapt during evaluation, which mirrors the setup in Yao et al. (2020). This means that if Agent A beats Agent B eighty percent of the time, that difference reflects Agent A being the stronger agent and not Agent B happening to play unusually well or poorly across those games. Additional metrics tracked per game included average game length in turns, average final hand score for each agent, and knock accuracy. Knock accuracy is defined as the percentage of games in which the agent chose to knock and subsequently won, accounting for the final turn each opponent receives after a knock. This metric directly evaluates the quality of the agent's knock timing decision independent of games where the opponent knocked first. For agents that maintain a belief state, the evaluation also tracks expected value error, the difference between the agent's predicted expected hand value at game end and the actual revealed hand score, as a measure of belief state accuracy. Games were capped at five hundred turns to prevent infinite loops in edge cases.

Results

The evaluation results across five hundred games per matchup are summarized in Table 1 below. Win percentages are reported for the first-listed agent in each matchup

Table 1. Head-to-head win rates across 500 games per matchup.

Matchup (A vs. B)	Agent A Win %	Avg Game Length	Avg Agent A Score	Avg Agent B Score
Heuristic vs. Random	56.6%	36.0 turns	23.39	26.01
POMDP vs. Random	79.0%	28.1 turns	18.71	27.47
POMDP vs. Heuristic	81.6%	69.1 turns	13.51	22.10
MC+POMDP vs. Random	84.0%	33.8 turns	17.41	27.89
MC+POMDP vs. Heuristic	93.6%	45.9 turns	11.78	24.81
MC+POMDP vs. POMDP	95.4%	81.2 turns	12.94	27.86

Table 2. Knock accuracy and belief state error for agents that use a belief state.

Matchup (A vs. B)	Agent A Knock Accuracy	Agent A EV MAE
POMDP vs. Random	91.3%	3.13
POMDP vs. Heuristic	83.2%	1.36
MC+POMDP vs. Random	89.4%	3.11
MC+POMDP vs. Heuristic	93.8%	1.40
MC+POMDP vs. POMDP	96.7%	1.48

The results confirm the expected ordering of agents, each agent beats those below it in the hierarchy, but the improvements are not uniform across levels. The heuristic agent's 56.6% win rate against the random agent is only marginally above chance, and its average final score of 23.39 versus the random agent's 26.01 reflects a small improvement in hand quality. The

heuristic agent adds measurable value over random play, a 56.6% win rate rather than 50%, but the gap is small, likely because it handles special actions (peek, swap, kingpin) randomly and has no memory of cards it has not directly drawn. The basic POMDP agent's 79.0% win rate against the random agent is a substantially larger jump, and its average final score of 18.71 compared to the random agent's 27.47 shows that belief-state reasoning produces significantly stronger hands. The POMDP agent also ends games in fewer turns against the random agent (28.1 turns) than the heuristic agent does (36.0 turns), suggesting the POMDP agent knocks earlier and more efficiently.

The MC+POMDP agent's results are the most striking. Its 93.6% win rate against the heuristic agent and 95.4% win rate against the basic POMDP agent demonstrate that the Monte Carlo layer adds substantial value on top of the belief state alone. The MC+POMDP agent's average final score of 11.78 against the heuristic agent is the lowest of any matchup, indicating it consistently achieves very strong hands before knocking. The gap between POMDP (81.6% vs. heuristic) and MC+POMDP (93.6% vs. heuristic) shows that using Monte Carlo rollouts to decide which card to draw is meaningfully better than the POMDP agent's approach of comparing the discard card's value against the current worst slot in a single step. The 95.4% win rate of MC+POMDP over POMDP is particularly notable because both agents share the same belief state and use identical logic for all actions except draw and knock, which are the two decisions where MC+POMDP substitutes Monte Carlo simulation for the POMDP agent's fixed heuristics.

Average game length also reveals an interesting pattern. Games between two strong agents (MC+POMDP vs. POMDP at 81.2 turns, POMDP vs. Heuristic at 69.1 turns) run significantly longer than games against the random agent (28.1–33.8 turns). This is because stronger opponents force the agent to wait for a more decisive advantage before knocking, whereas against the random agent the agent can knock relatively quickly.

The knock accuracy results in Table 2 directly evaluate the quality of the knock timing decision. The POMDP agent knocks with 91.3% accuracy against the random agent and 83.2% against the heuristic agent, meaning that when it decides to end the round, it has the lower score the large majority of the time. The MC+POMDP agent improves on this in harder matchups, reaching 96.7% knock accuracy against the basic POMDP agent. The lower knock accuracy against the heuristic agent (83.2% for POMDP, 93.8% for MC+POMDP) compared to the random agent suggests that the heuristic agent occasionally builds a competitive hand before the opposing agent knocks, which is the same characteristic that gives it a 56.6% win rate over the random agent.

The belief state error analysis shows that the expected hand value estimated by the POMDP belief state is a reasonably accurate predictor of the true hand score at game end. Against the random agent, where fewer cards are ever revealed through peeks, the mean absolute error is higher (3.13 for POMDP, 3.11 for MC+POMDP), since more slots remain unobserved throughout the game. Against the heuristic agent, the EV MAE drops to 1.36–1.48, reflecting that the POMDP agent peeks more aggressively in longer games and accumulates more accurate slot-level information.

Conclusion

This project successfully implemented a full game environment for Stool Pigeon and four AI agents of increasing sophistication, and demonstrated through systematic evaluation that each additional layer of reasoning, from random to heuristic to POMDP to MC+POMDP, produces measurable improvements in win rate against all weaker baselines.

The POMDP belief state design was one of the more technically involved parts of the project. Tracking uncertainty correctly across all game events, especially Bamboozle swaps where known cards move between slots and some certainty is lost, required careful

implementation of the belief update logic. The unified observation system, where each game event creates a typed observation object that applies its own update to the belief state, made this manageable and kept the belief update logic cleanly separated from the decision logic.

The Monte Carlo simulation layer was the other major design challenge. Deep copying the full game state for each simulation is expensive, and the rollout policy had to cover every possible action phase to prevent simulations from getting stuck. The one hundred simulations per draw decision was chosen as a balance between computational cost and estimated stability. In practice the MC+POMDP agent takes noticeably longer per turn than the POMDP agent, which is visible in the GUI mode but acceptable for a two-player card game.

There are several limitations to the current approach. The Monte Carlo layer in the MC+POMDP agent is applied to both the draw and knock decisions. However, for keep, swap, peek, and kingpin decisions, the basic POMDP heuristics are still used, which means the agent may still make suboptimal decisions in those phases. The most promising extension would be applying Monte Carlo simulation to swap decisions triggered by Bamboozle and Vendetta. Swaps have an outsized strategic impact because they permanently move cards between crime scenes. The current heuristic, select whichever swap maximizes the difference between the expected value of the slot you give away and the slot you receive, is a one-step calculation that does not account for how the swap interacts with future draws or knock timing. A simulation-based approach could evaluate specific swap targets by rolling out the rest of the game from the resulting state, potentially finding swaps that set up a faster or more decisive knock. Peek decisions are less likely to benefit from MC since targeting the opponent's highest-uncertainty slot is already near-optimal as a one-step rule.

Another limitation is that the belief state uses a uniform prior over the unknown card pool, treating all unseen cards as equally likely. In reality, an opponent's actions carry information, like if an opponent keeps a drawn card, they probably drew something low-valued or if they discard it, they likely drew something high. Yao et al. (2020) incorporates this kind of

behavioral signal directly into their Bayesian belief updates by conditioning on the opponent's observed actions, and applying a similar approach to Stool Pigeon is a natural area for future improvement. The current observation system only updates beliefs based on directly visible cards, not on inferred information from opponent behavior.

A third limitation involves the draw pile reshuffle mechanic. When the draw pile is exhausted, the discard pile is reshuffled back into it. The belief state is not notified when this happens, so cards that were previously marked as removed from the game remain absent from the unknown pool even though they are now drawable again. This causes the belief state's expected value estimates to become slightly inaccurate after a reshuffle. In practice this is a rare edge case given average game lengths of 28–81 turns, but it is a known gap in the belief state's accuracy that could be addressed by introducing a reshuffle observation type that restores the relevant cards to the unknown pool.

Despite these limitations, the MC+POMDP agent achieves a 95.4% win rate against the basic POMDP agent and 93.6% against the heuristic agent, which represents a strong result for a card game with the complexity of Stool Pigeon. Unlike the poker setting in Yao et al. (2020), where only the opponent's hand is hidden, Stool Pigeon requires the agent to maintain belief states over both players' hidden cards simultaneously. Also, with the variety of special action cards in Stool Pigeon, each distinct information event must be tracked, representing a more complex observation structure than the betting patterns used in poker. The project demonstrates that the POMDP and Monte Carlo framework of Yao et al. (2020) generalizes well beyond poker to a game with mutual hidden information, multiple special card types, and a dynamic knock-timing decision, and that the resulting agents produce meaningful and measurable improvements over simpler baselines.

Works Cited

- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Miller, N. (2023). Stool Pigeon [Card game]. Barrel Aged Games.
- Python Software Foundation. (2024). *multiprocessing — Process-based parallelism* (Python 3.x documentation). <https://docs.python.org/3/library/multiprocessing.html>
- Python Software Foundation. (2024). *Python* (Version 3.11.4) [Software]. <https://www.python.org>
- Pygame Community. (2024). *Pygame* (Version 2.6.1) [Software]. <https://www.pygame.org>
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Yao, J., Zhang, Z., Xia, L., Yang, J., & Zhao, Q. (2020). Solving Imperfect Information Poker Games Using Monte Carlo Search and POMDP Models. 2020 IEEE 9th Data Driven Control and Learning Systems Conference (DDCLS). <https://doi.org/10.1109/ddcls49620.2020.9275053>